

P.E.D - PREPARATÓRIO PARA ENTREVISTAS EM DADOS

APOSTILA GIT E GITHUB



SUMÁRIO

Introdução	2
Como funciona o git?	3
O que é uma branch?	4
Boas práticas	5
Cheatsheets de comandos do Git	5
O que é um workflow?	7
Branches principais	8
Branches secundárias	8
Como usar o git?	9
Links úteis	10

INTRODUÇÃO

DANIELA XAVIER
daniela.xavier28@gmail.com



Caro aluno do PED,

Seja muito bem-vindo a nossa apostila de Git e Github. Este material foi elaborado pensando auxiliar você no uso de ferramentas de versionamento e colaboração.

O Git e o GitHub são fundamentais no mundo do desenvolvimento de software moderno, permitindo que equipes colaborem de maneira eficiente e organizada. Com esta apostila, nosso objetivo é oferecer uma fonte de aprendizado clara e estruturada, que sirva tanto como um recurso introdutório quanto um manual de consulta rápida para questões específicas.

Este material é um complemento às nossas aulas, proporcionando um reforço teórico e prático sobre os tópicos abordados. Aqui, você encontrará:

1. Explicações Detalhadas: Cada conceito é abordado com clareza para garantir uma boa compreensão.
2. Cheatsheets Práticos: Resumos e comandos essenciais do Git estão à disposição para consulta rápida.
3. Links Úteis: Referências adicionais para expandir seu conhecimento e habilidades.

DANIELA XAVIER
daniela.xavier28@gmail.com

Recomendamos que utilize este guia como uma ferramenta ativa, aplicando o que aprender aqui em seus próprios projetos e portfólio.

COMO FUNCIONA O GIT?



O Git é um sistema de controle de versão amplamente utilizado em desenvolvimento de software. Funciona como uma espécie de "pasta na nuvem", conhecida como repositório, mas com funcionalidades muito mais avançadas. Ao trabalhar com Git, você cria uma cópia local deste repositório, permitindo trabalhar em seu próprio ambiente. Cada mudança que você faz é registrada localmente e pode ser "empurrada" (upload) de volta para o repositório na nuvem.

Essa abordagem traz várias vantagens. Primeiramente, ela aumenta a organização do projeto, pois cada mudança é rastreada e pode ser revertida se necessário. O Git permite o trabalho colaborativo e em paralelo, onde múltiplas pessoas podem trabalhar em diferentes partes do código simultaneamente sem interferir umas nas outras. Além disso, cada alteração é registrada com detalhes sobre o autor e a data, facilitando o acompanhamento do progresso e a responsabilização.

Outro ponto forte do Git é a segurança e a integridade do código. As mudanças são rastreadas e armazenadas de forma segura, e o sistema possui mecanismos para resolver conflitos que podem surgir quando múltiplas pessoas estão trabalhando no mesmo projeto.

O QUE É UMA BRANCH?

No contexto do Git, uma "branch" ou ramificação, é um mecanismo fundamental para o versionamento de código e a colaboração em projetos de software. Ao contrário de ser uma simples cópia da "pasta" do Git, uma branch é, na verdade, um ponteiro móvel para um commit específico. Quando você cria uma branch, está criando uma linha de desenvolvimento independente. Isso significa que você pode fazer alterações no código, experimentar novas ideias ou desenvolver funcionalidades sem afetar a linha principal do projeto, geralmente chamada de "master" ou "main".

As branches são essenciais para uma organização eficiente e segura de projetos de software. Elas permitem que diferentes recursos ou correções de

bugs sejam desenvolvidos em paralelo, sem interferir um no outro. Por exemplo, enquanto uma equipe trabalha em uma nova funcionalidade em uma branch separada, outra equipe pode manter a branch principal estável e liberada para produção.

Além disso, a gestão de branches facilita o processo de revisão de código e integração de mudanças, uma vez que as alterações feitas em uma branch podem ser revisadas e mescladas (merge) de volta à branch principal. Isso garante que o código em desenvolvimento não "quebre" o projeto principal e permite que várias pessoas colaborem eficientemente no mesmo código.

BOAS PRÁTICAS

Boas práticas são fundamentais em qualquer área de atuação, mas ganham uma importância ainda mais crítica no campo do desenvolvimento em conjunto. Adotar práticas padronizadas e eficientes não só melhora a qualidade do código, mas também facilita a colaboração, a manutenção e o escalonamento de projetos de software.

DANIELA XAVIER
daniela.xavier28@gmail.com

Um exemplo proeminente de boas práticas no desenvolvimento de software é o uso de "Commits Convencionais". Esta é uma especificação para adicionar significado legível às mensagens de commit. Indicamos que seja seguido esse padrão. Suas regras, exemplos e especificações estão disponíveis em <https://www.conventionalcommits.org/en/v1.0.0/#specification>.

CHEATSHEETS DE COMANDOS DO GIT

Configuração Inicial

- git clone
 - Descrição: Copia um repositório Git.
 - Uso: git clone [url-do-repositório]
 - Exemplo: git clone https://github.com/exemplo/repositorio.git

Trabalhando com Branches

- git checkout [nome-da-branch]
 - Descrição: Troca para a branch especificada.
 - Uso: git checkout [nome-da-branch]
 - Exemplo: git checkout desenvolvimento
- git checkout -b [nome-da-nova-branch]
 - Descrição: Cria uma nova branch e troca para ela.
 - Uso: git checkout -b [nome-da-nova-branch]
 - Exemplo: git checkout -b recurso-novo

Atualizações e Mudanças

- git fetch
 - Descrição: Atualiza suas referências locais com as remotas.
 - Uso: git fetch
- git pull
 - Descrição: Atualiza sua branch local com a versão mais recente do repositório remoto.
 - Uso: git pull
- git status
 - Descrição: Apresenta o status atual da sua branch, incluindo mudanças não rastreadas ou pendentes.
 - Uso: git status

Gerenciamento de Mudanças

- git add .
 - Descrição: Adiciona todas as mudanças atuais ao stage para o próximo commit.
 - Uso: git add .
- git commit -m "[mensagem]"
 - Descrição: Cria um commit com as mudanças que foram adicionadas ao stage.

- Uso: git commit -m “[mensagem]”
 - Exemplo: git commit -m “Adiciona nova funcionalidade”
- git push
- Descrição: Envia as alterações commitadas para o repositório remoto.
 - Uso: git push

Fusões e Exclusões

- git merge [nome-da-branch]
- Descrição: Incorpora as mudanças de uma branch especificada na branch atual.
 - Uso: git merge [nome-da-branch]
 - Exemplo: git merge recurso-novo
- git branch -D [nome-da-branch]
- Descrição: Apaga uma branch localmente.
 - Uso: git branch -D [nome-da-branch]
 - Nota: Tenha cuidado para não apagar a branch errada.
 - Exemplo: git branch -D recurso-antigo

O QUE É UM WORKFLOW?

Workflow é um conjunto de regras que define como as branches de um repositório Git devem ser criadas, nomeadas, utilizadas e integradas. Ele vai além da simples organização de branches, pois dita a maneira como as equipes colaboram no desenvolvimento de software, garantindo uma metodologia consistente e eficiente para a integração de mudanças no código.

Essencialmente, um workflow determina o processo pelo qual as mudanças são feitas, revisadas, testadas e, finalmente, incorporadas ao produto final. Isso inclui não apenas o gerenciamento de branches, mas também práticas para revisão de código (como pull requests), estratégias de teste e integração, e diretrizes para lançamento de novas versões.

O uso de um workflow padronizado traz várias vantagens. Primeiramente, ele assegura a segurança e integridade dos projetos, pois cria uma barreira contra mudanças inesperadas ou mal-intencionadas no código principal. Além disso, facilita o versionamento e a manipulação do histórico de commits, permitindo que as equipes rastreiem eficientemente o progresso do projeto e revertam para versões anteriores se necessário.

Também, ao adotar um workflow, as equipes podem colaborar de maneira mais organizada e previsível. Isso é particularmente importante em projetos grandes ou em ambientes onde várias equipes ou contribuidores estão trabalhando simultaneamente em diferentes aspectos do mesmo projeto.

BRANCHES PRINCIPAIS

Master

- É a branch de produção. Sempre representa a versão mais estável do software (versão que está disponível ao público).

DANIELA XAVIER
daniela.xavier28@gmail.com

Develop

- É onde novas features vão se acumulando, oferecendo ao time de desenvolvimento uma visão de quais features já foram implementadas por outros desenvolvedores, e testar como sua feature se comunicará com as demais. Por ser utilizada com maior frequência por códigos ainda em desenvolvimento, é mais suscetível a instabilidades.

Homolog

- É onde as features com seu desenvolvimento já finalizado, e já testadas pelo time de desenvolvimento se acumulam, aguardando a homologação (aprovação) da liderança. Esta é a branch responsável por entregar as features para a branch de produção.

BRANCHES SECUNDÁRIAS

Feature

- É onde o programador irá desenvolver as funcionalidades do programa.

Ela sempre deve derivar da Master, com isto, é garantido que sempre estará sendo realizado o desenvolvimento em cima de uma versão estável. Uma vez que o desenvolvimento esteja finalizado, as informações devem ser enviadas para develop, e após testadas, para a delivery branch.

Delivery

- Tem como intuito “limpar” o histórico de commits na homolog, facilitando sua manipulação. Sendo assim, pense na Delivery como a feature branch, porém com apenas 2 commits (a master e um compacto das novas funcionalidades). Este commit compactado será resultante do merge squash.

COMO USAR O GIT?

Aqui está um guia passo a passo para começar a usar o Git:

DANIELA XAVIER

daniela.xavier28@gmail.com

1. Instalação do Git:

O primeiro passo é instalar o Git em seu computador. Isso pode ser feito acessando Git Downloads, onde você encontrará versões do Git para diferentes sistemas operacionais. Siga as instruções de instalação específicas para o seu sistema.

2. Configuração Inicial:

Após a instalação, é importante configurar suas credenciais no Git. Isso identifica você como autor de suas contribuições nos repositórios. Abra um terminal ou prompt de comando e configure seu nome e e-mail:

- `git config --global user.name "Seu Nome"`
- `git config --global user.email "seu-email@exemplo.com"`

3. Acesso ao Repositório:

Para trabalhar em um projeto, você precisa ter acesso ao repositório Git do mesmo, normalmente hospedado em plataformas como GitHub, GitLab ou Bitbucket. Certifique-se de ter acesso ao repositório verificando na seção de repositórios da plataforma escolhida.

4. Clonando o Repositório:

Uma vez que você tenha acesso, clone o repositório para criar uma cópia local no seu computador. Isso permite que você trabalhe no código sem afetar a versão principal. No terminal, navegue até o diretório onde deseja que o repositório seja clonado e execute:

- `git clone [URL do Repositório]`

5. Trabalhando com o Repositório:

Após clonar, você pode começar a trabalhar no repositório. Isso geralmente envolve criar branches, fazer alterações, commitar essas alterações e empurrá-las (push) de volta para o repositório remoto. Lembre-se de seguir o workflow estabelecido pelo seu projeto ou equipe, que pode incluir práticas para revisão de código, criação de pull requests, e estratégias de teste.

6. Colaboração e Atualizações:

Enquanto trabalha em um projeto colaborativo, é importante manter seu repositório local sincronizado com as mudanças feitas pelos outros membros da equipe. Regularmente, faça git pull para atualizar seu repositório local.

DANIELA XAVIER
daniela.xavier28@gmail.com

7. Compartilhando Suas Mudanças:

Quando suas alterações estiverem prontas, compartilhe-as com a equipe fazendo commits e usando git push para enviar suas mudanças para o repositório remoto.

LINKS ÚTEIS

<https://git-scm.com/downloads>

<https://docs.github.com/en>

<https://www.conventionalcommits.org/en/v1.0.0/#summary>

<https://medium.com/the-click-reader/git-for-data-science-a-guide-for-data-scientists-9cedf9f550e5>

<https://github.com/ByteByteGoHq/system-design-101#how-git-commands-work>

LISTA DE EXERCÍCIOS

1 - Você vai começar a trabalhar num projeto que já está em andamento, o qual possui um repositório de git associado. Considere as seguintes circunstâncias:

- Atualmente esse repositório contém dois branches: master e refactor (este último derivado de master).
- O branch no qual você trabalhará, feature, ainda não existe. Neste branch você gostaria de considerar tudo o que já foi implementado no branch refactor.
- Você acabou de clonar o repositório e, portanto, está trabalhando (está checkout) em master.

Qual das sequências de comandos abaixo seria a mais apropriada para criar e começar a trabalhar no branch feature?

- a) git branch feature; git pull refactor
- b) git checkout -b feature; git branch refactor
- c) git checkout refactor; git branch feature; git checkout master
- d) git checkout refactor; git checkout -b feature
- e) git branch -b refactor; git branch feature

2 - Enquanto você trabalhava no seu branch feature as alterações de refactor ficaram prontas para serem mergeadas em master. Seu colega, que estava encarregado do branch refactor, pediu para que você fizesse o merge para ele. Considere o seguinte cenário:

- Na sua máquina os branches locais refactor e master já estão atualizados com as últimas modificações feitas nos respectivos branches remotos.
- Atualmente você está trabalhando no seu branch feature.

Qual das sequências de comandos abaixo seria a mais apropriada para iniciar o processo de levar as alterações do branch refactor ao branch master?

- a) git merge master; git checkout refactor
- b) git checkout refactor; git merge master
- c) git checkout master; git merge refactor
- d) git merge refactor; git checkout master
- e) git merge refactor; git checkout refactor

RESPOSTAS

1 - A sequência correta de comandos para criar e começar a trabalhar no branch **feature**, considerando que você quer que este branch tenha como base tudo que foi implementado no branch **refactor**, é:

DANIELA XAVIER

- **git checkout refactor**: Este comando muda para o branch **refactor**, que é o ponto de partida para o novo branch **feature**.
- **git checkout -b feature**: Depois de mudar para o branch **refactor**, este comando cria e muda para um novo branch chamado **feature**, que conterá todos os commits do branch **refactor**.

2 - Para iniciar o processo de levar as alterações do branch **refactor** ao branch **master**, você precisará primeiro certificar-se de que está no branch **master** e depois mesclar as alterações do branch **refactor** para dentro dele. A sequência correta seria:

1. **git checkout master**: Muda para o branch **master**.
2. **git merge refactor**: Mescla as alterações do branch **refactor** no branch **master** atual.